PROJECT FIELD GUIDE

CyberShield VerifyPay

Technology, feasibility & how the application works

A plain-language guide to the codebase, its moving parts, local development, hosting, and current limits.

At a glance

WHAT IT IS	A signed-in web app for reviewing payment messages, links, and images for explainable warning patterns.
	OCR reads image text; transparent rules look for warning patterns and produce a preliminary signal.
WHAT IT IS NOT	It does not verify a payment, identify an account owner, or establish that a message or website is safe.

This guide reflects the repository documentation and source files reviewed for this project. It explains the current implementation; it does not certify a security review or confirm that a hosted deployment is live.

01 | The technology and request flow

The app is split into a website, an API server, and a database. Each has a different job. The browser displays the interface; the API validates requests and runs application logic; PostgreSQL keeps the records that should persist.

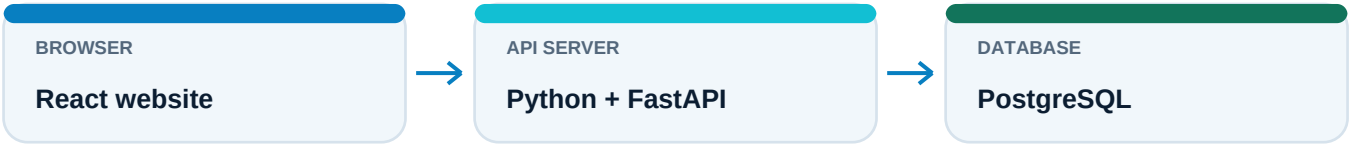

IMAGE CHECK Image → Tesseract OCR → text-pattern rules → preliminary result

LAYER	TECHNOLOGY	ROLE IN THIS PROJECT
Website	React, JavaScript/JSX, CSS; Vite and npm	Renders screens, manages page state, and calls the API. Vite runs the local development website and builds static files for hosting.
API	Python, FastAPI, Uvicorn, Pydantic	Provides endpoints for accounts, checks, history, trusted payees, and related actions; validates incoming and outgoing data.
Data	PostgreSQL, SQLAlchemy, psycopg	Stores account records, saved check summaries, and trusted payees. Alembic migration files track schema changes.
Image reading	Tesseract OCR, pytesseract, Pillow	Checks and prepares uploaded images, then extracts text for the same pattern analysis used on typed content.
Identity & email	Password/session helpers; Authlib for Google OIDC; SMTP or Brevo	Supports account sign-in flows and email verification/reset. Provider credentials and mail settings must be configured per environment.

Data handling in the current design

Project documentation says raw submitted payment text and OCR output are not stored in PostgreSQL; a result summary is saved for the signed-in account. OCR text may be included in the immediate API response so the user can inspect what was read. Browser-local notifications are stored in that browser rather than on the server.

02 | Feasibility and honest limits

A small demo for a known group is technically achievable with this stack. The core product path exists: submit content, analyze it, show the reasons, and save account-scoped summaries. Hosting still requires a running API and database, plus correct environment settings.

What the result means

The risk score is a rules-based pattern signal, not a probability of fraud. The app does not contact the recipient, visit submitted links, look up live domain reputation, or verify transfers. OCR quality depends on image clarity, language, and layout. A missing amount or recipient can mean the text was not recognized or the extraction rules did not match the wording.

CAPABILITY	CURRENT BEHAVIOR / LIMIT	PRACTICAL CONSEQUENCE
Payment analysis	Explainable keyword and pattern rules; no trained fraud model.	Useful for demonstrations and prompts to double-check; not a payment safety guarantee.
Link analysis	Inspects submitted URL text; does not open or reputation-check the site.	A warning or no warning cannot establish that a website is legitimate.
Image extraction	Tesseract OCR reads the image before text rules run.	Review extracted text and manually enter details when the image is unclear.
Private audience	Hosting produces a network-accessible site; a link alone is not invite-only.	Restrict account creation/access before sharing if only selected people should use it.
Notifications	Payment-check notifications are local to the current browser.	They are not server-side push, SMS, or email notifications.

Hosting layout

The Render Blueprint and backend Dockerfile describe a path where the built website and API are served together by one service, with PostgreSQL hosted separately. Netlify and Vercel can serve the built React website, but they do not replace this Python API or its database. In that split setup, the frontend must be configured to call the API’s public HTTPS address, and the API must allow the frontend origin.

Deployment status

Configuration files exist, but that alone does not mean deployment succeeded. A real hosted launch is confirmed only after the API health check, database connection, sign-in/email settings, frontend-to-API calls, and a full user flow work at the hosted URLs. This review did not confirm a live deployment.

03 | How you work on it

A normal change moves through a short loop: understand the existing behavior, make a focused edit, run the affected checks locally, review the result, then save the change with Git. Database edits need an additional migration review.

STEP	WHAT YOU DO	WHY IT MATTERS
1. Edit	Use VS Code to change the relevant React, Python, style, or configuration files.	Keep a change focused so it is easier to understand and reverse.
2. Run locally	Start PostgreSQL, the FastAPI backend, and the Vite frontend; keep their terminals open.	The browser app depends on both the API and database for most real workflows.
3. Verify	Use the browser to exercise the changed workflow and run the existing backend tests when relevant.	A page loading is not proof that API and database workflows work.
4. Migrate carefully	For a schema change, review the Alembic migration and database baseline before applying it.	A migration changes persistent database structure; applying it to the wrong baseline can break a database.
5. Save with Git	Review changed files, then commit and push when you decide the result is ready.	Git records project history and makes sharing/deployment from the repository possible.
6. Host & check	Build the app, set environment values in hosting dashboards, deploy, then test hosted URLs.	Local success and hosted success are separate checks.

Local setup notes

The README describes Windows setup: Python 3.12+, Node.js/npm, PostgreSQL, and Tesseract OCR with English language data. The frontend uses Vite at a local address such as http://localhost:5173. The backend is a separate Uvicorn process. Check the actual backend port and the Vite proxy target if you see connection-refused errors: the checked-in Vite default is 8000, while the README's backend command uses 8001; a local environment setting may override the default.

Keep private settings private

Database URLs, passwords, OAuth secrets, email credentials, and encryption keys belong in private environment files locally or in the host's secret settings. The example environment files document variable names. Never put real values in Git or screenshots.

04 | Quick glossary and readiness

Plain-language definitions

TERM	PLAIN MEANING
Frontend	The part people see and use in their browser.
Backend / API	The server that receives requests, checks them, and returns results.
Database	The organized storage used to keep information between visits.
Environment variable	A setting supplied when the app runs, often used for URLs or private credentials.
Migration	A reviewed, versioned instruction for changing database structure.
OCR	Optical character recognition: software that reads text from an image.
Build	The deployable website files generated from the frontend source.

Practical readiness snapshot

AREA	CURRENT UNDERSTANDING
Prototype / demo	Feasible with the existing stack; validate the main workflows and hosted configuration before sharing.
Sensitive production use	Not established by this project review. The app documentation describes it as an educational prototype and warns against sensitive payment data.
MFA	MFA implementation is present in the codebase; the user has asked to leave MFA work until last.
Microsoft sign-in	Deferred in the project plan.
Live deployment	Not verified in this review.

Repository references

README.md · docs/ARCHITECTURE.md · frontend/package.json · frontend/vite.config.js · backend/requirements.txt · backend/app/main.py · backend/app/risk.py · backend/app/ocr.py · render.yaml

The safest way to keep moving	Treat each change as a small cycle: edit → run → check → save. Keep the score's limits visible, protect account data, and confirm the hosted flow before inviting people.